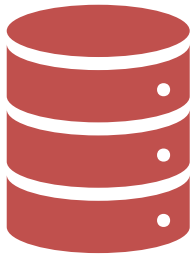


Networks and Systems Management



Repositories and CI/CD
Filipe Sousa
FEUP – GRS/M.EIC

What Is a Repository?



A repository is a centralized place to store and manage code and related files.

Key features:

Version control (e.g., Git)

Collaboration and change tracking

Branching and merging

Access control and audit trails



Popular platforms:

GitHub

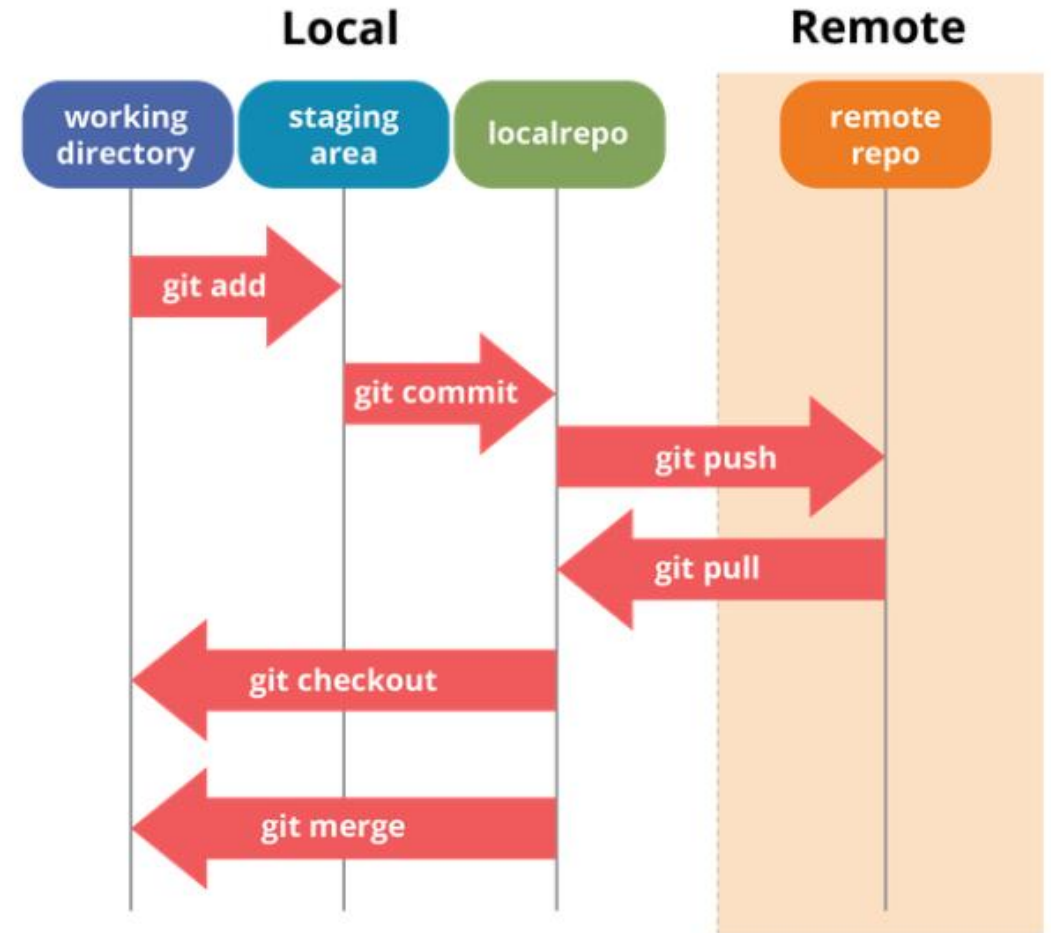
GitLab

Bitbucket

Git Basics (Quick Recap)

Essential Git concepts:

- clone, add, commit, push, pull
- Branches: develop in isolation
- Merge/Pull Request (PR): controlled integration
- Tags: mark versions/releases



Introduction to CI/CD

1

CI/CD = Continuous Integration / Continuous Delivery (or Deployment)

2

Continuous Integration:
Automatically testing and validating code as it's committed.

3

Continuous Delivery:
Automatically preparing code for release.

4

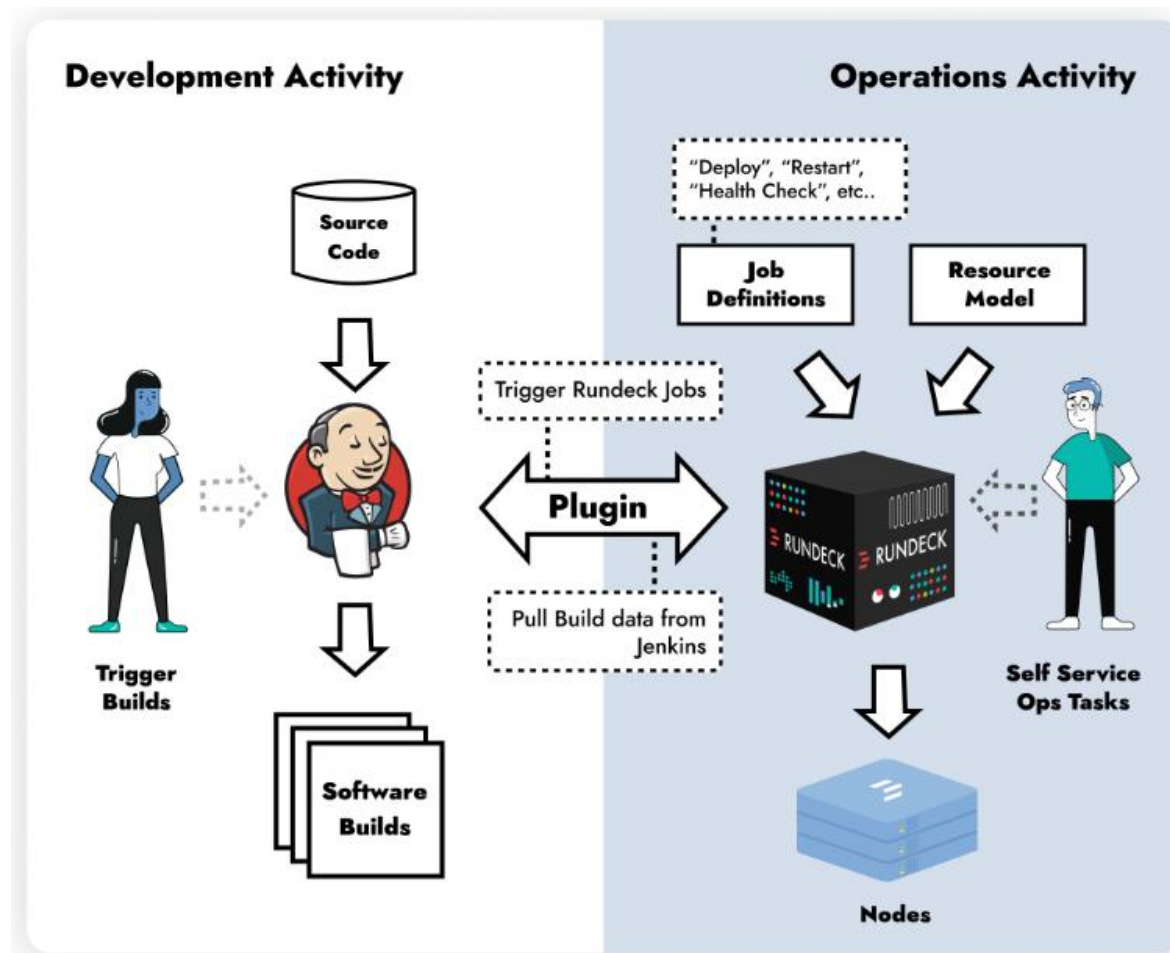
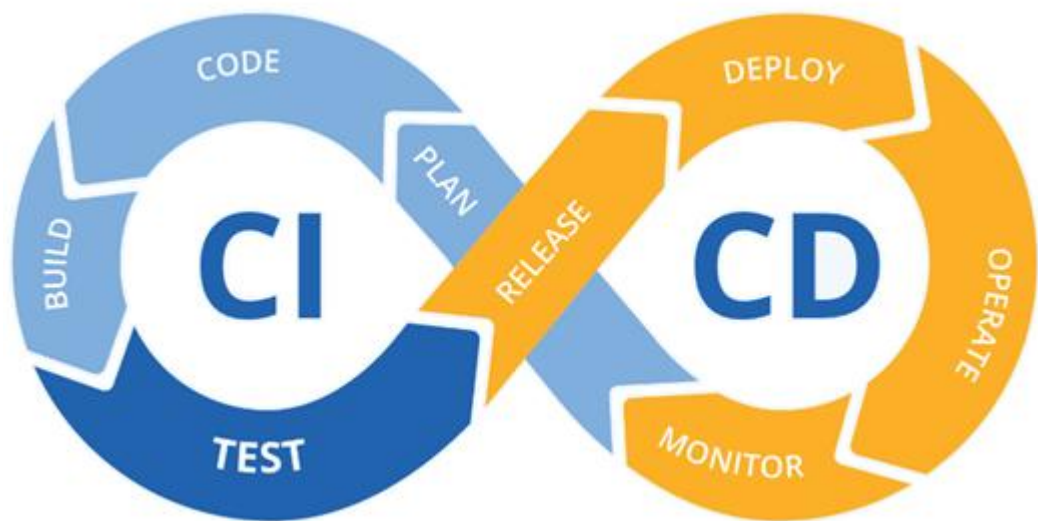
Continuous Deployment:
Automatically releasing validated code to production.

CI/CD Pipeline Overview

Stages:

1. **Source:** Code pushed to repository
2. **Build:** Compilation and packaging
3. **Test:** Unit, integration, and system tests
4. **Deploy:** To staging or production
5. **Monitor:** Observability and feedback

CI/CD Pipeline



Code vs. build

Straightforward distinction, e.g., in C or Java ... or when you package repositories

- Source code vs. executable binaries

Need a build phase for infrastructure?

- Depends on which language you write the infrastructure code and tools
- What you're deploying are configuration files

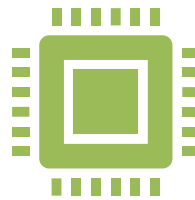
Test



Unit test

Sandbox or dry run tests of individual components / configurations

Some networking equipment has this option



System test

Deploy the different network components and test their interactions

Testing environment? Network emulator, virtualization



Staging environment

Replica of production environment

Test specifics of production environment (number of nodes, addresses, etc.)

Release vs. Deploy



A RELEASE IS THE SOFTWARE
PRODUCT, READY TO BE DEPLOYED



A DEPLOYMENT IS A RELEASE
CONFIGURED FOR THE TARGET
ENVIRONMENT



WHAT'S THE DIFFERENCE FOR
INFRASTRUCTURE?

Abstractions for Infrastructure Code



Target: specific deployment of infrastructure



Abstraction 1:

Function that takes parameters in and outputs configuration files

How do the configuration files end up in the infrastructure?



Abstraction 2:

Code that takes care of generating and deploying configuration files

Based on parameters from the user

Popular CI/CD Tools

GitHub
Actions

GitLab CI/CD

Jenkins

Travis CI

CircleCI

ArgoCD / Flux
(for GitOps)

Benefits of CI/CD

Faster delivery of features and fixes

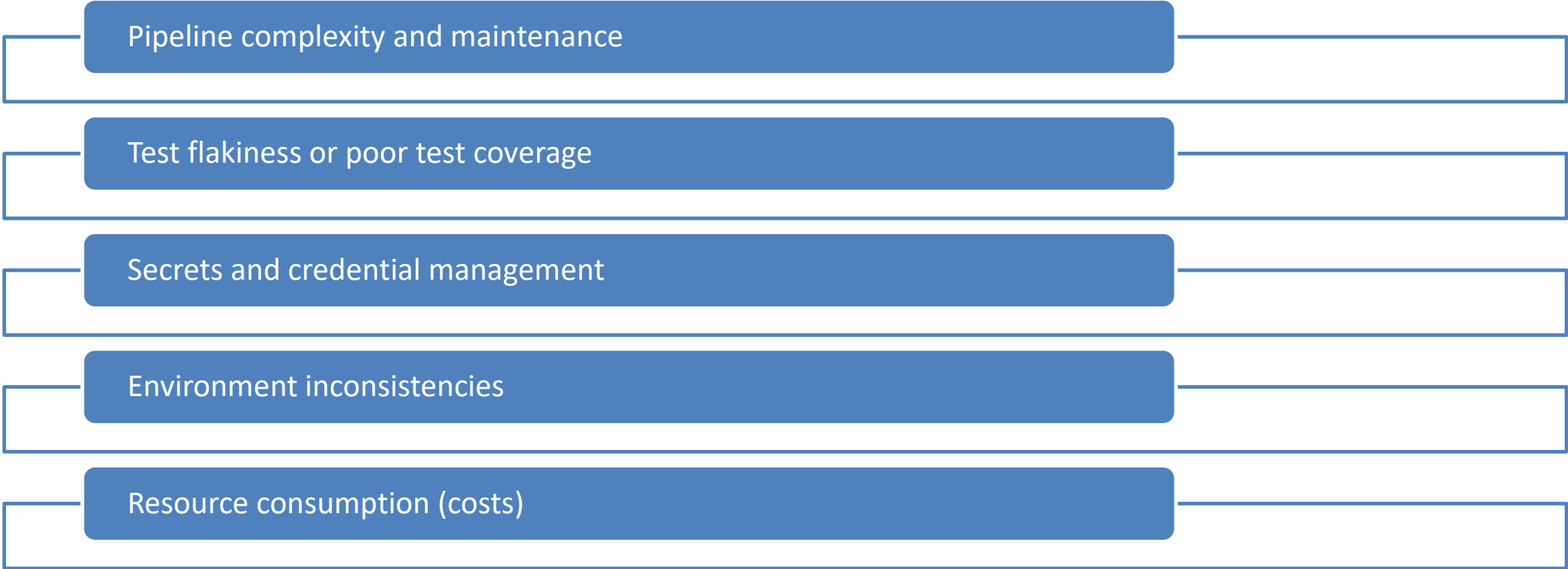
Early detection of bugs

Improved collaboration and integration

Reduced manual deployment errors

Repeatable and traceable processes

Common Challenges



Pipeline complexity and maintenance

Test flakiness or poor test coverage

Secrets and credential management

Environment inconsistencies

Resource consumption (costs)

Best Practices



Keep builds fast
and tests relevant



Use environment
variables securely



Version
infrastructure (IaC)
alongside app code



Separate
dev/staging/prod
environments



Monitor pipeline
health and
performance



Automate rollback
and failure
notifications

Example



NETCONF communication with devices



Ansible with NETCONF module for automation



GitHub as repository



Jenkins for automating CI/CD



Operations

Build: gets facts from deployment, validates configurations

Test: dry run-in target devices (JunOS, commit-check)

Deploy: see if there are changes pending (config vs state of devices), only deploy changes

https://archive.nanog.org/sites/default/files/monday_general_autobuild_vicente_63.28.pdf

Key Takeaways

Repositories enable structured, collaborative development.

CI/CD pipelines ensure code quality and accelerate delivery.

Automation reduces manual error and increases reliability.

Choose the right tools and practices for your context.

Security and monitoring are integral to CI/CD success.

Further Reading & References

- [Git Documentation](#)
- [GitHub Actions Docs](#)
- [GitLab CI/CD Docs](#)
- [Jenkins Documentation](#)
- [Martin Fowler on CI/CD](#)

